

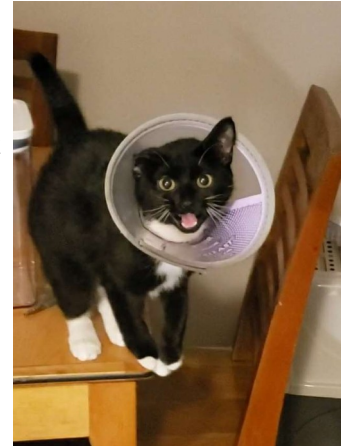
A Crash Course in Arduino Programming

Lesson 2

A. Introduction

So, you've survived lesson 1.... Good for you! Now the real fun can begin! If you are following along in person, you just need to bring an Arduino, your laptop, and an Arduino cable. If you are following along with the handout, but not with the in person class, look ahead at each example exercise to see what additional circuit supplies you'll need.

In the image to the right, you can see a Boolean¹ (the author's cat) attempting to perform an operation (receive radio signals??). This leads us into our first topic: Boolean logic!



B. Boolean Logic

Boolean logic asks questions that can have one of two answers: true or false. Boolean logic is foundational to digital computing, because the fundamental unit of information (the bit) is binary. It can be one of two numbers: 1 or 0. So, Boolean logic is often utilized to determine whether a bit should be 0 or 1, with 1 typically corresponding to true and 0 typically corresponding to false. Later in this lesson, we will be using boolean logic to check whether **conditions** have been met to either perform a certain operation (as with if else and switch case) or to exit a loop (as with while and for).

The fundamental operations of boolean algebra are AND, OR, and NOT. There are several ways to notate these operations, so I will show a few common examples for each.

A AND B: A&B, A&&B, AB, A*B

A OR B: A|B, A||B, A+B

NOT A: !A, ~A, $\overline{A}$

We summarize how boolean operations (or combinations of operations) behave with **truth tables**.

A	B	A AND B
False	False	False
False	True	False
True	False	False
True	True	True

¹ Her full name is "Boolean Undeclared-in-This-Scope Ray."

A	B	A OR B
False	False	False
False	True	True
True	False	True
True	True	True

A	NOT A
False	True
True	False

Boolean Operators

Logical AND	&&
Logical OR	
Logical NOT	!

Comparison operators

Equal to	==
Not equal	!=
Less than	<
Less than or equal to	<=
Greater than	>
Greater than or equal to	>=

You may be wondering why we say “Logical AND” instead of just “AND” or why we use “&&” instead of just one. There is another set of operators called **bitwise operators** that can do similar operations but need to be distinguished from logical operators. We will not be using these operators in today’s lesson, but I will list them for you here just in case you run into them.

Bitwise Operators

Bitshift left	<<
Bitshift right	>>
Bitwise AND	&
Bitwise OR	
Bitwise NOT	~
Bitwise XOR	^

C. Functions

Functions are units of code that perform a specific task. Functions have a **return type**, a name, **parameters**, and **scope**.

1. Function Syntax

Creating a function:

```
<return type> <function name>(<parameter type> <parameter name>, ...)
{
    // do stuff here
    return <return value>
}
```

Calling a function:

```
<return type> <variable name> = <function name>(<argument1>, ...);
```

If there is no return value:

```
<function name>(<argument1>, <argument2>, ...);
```

2. Function Conventions

In Arduino, there is no *one* correct way of naming your functions. However, it is important to BE CONSISTENT² once you pick a naming style. Generally, folks will use either **snake_case** or **camelCase**. Occasionally you will see **PascalCase** for Arduino function names, but it is less common. Again, pick one, and *be consistent*³.

3. Return

You'll notice that the last thing we do in many of the function examples shown is **return** a value. The return keyword terminates the function and returns the value specified after it to the calling function. It is not necessary to use the return keyword in your function if it has no return type (or a void return type), and return does not always have to be used in combination with a value to deliver to the calling

² Try to be consistent.

³ Did I mention you should be consistent?

function. A return can be used anywhere in your function, not just the end, but you must be mindful of the fact that after the line where the return is executed, the rest of the function will not run.

4. Scope

Generally, variables created within a function, are only visible to that function. If you try to access the variable outside of that function, you will get an error because you are outside of the **scope** in which the variable is defined. This applies to void setup, void loop, and, as you'll see later in this lesson, loops also have a scope! What if we want to make a variable that can be accessed anywhere in the code? That's called a **global variable**; these variables need to be created at the top of the code, outside of anything with curly-brackets ({}).

5. Documentation and Comments

Like naming styles, documentation and commenting style varies from programmer to programmer. The important thing is that you have clear documentation and that you are... dare I say it... consistent! Here is an example of my typical style for describing functions inside of the program.

```
/*
A Descriptive Title
Author: Your Name Here
Date: month day, year

Put a description of what your program does here.
*/

void my_function(int arbParam) {
    /*
    my_function
    - Put a description of what your function does here
    Arguments:
    - (<type>) <variable name> - description
    - (int) arbParam - an arbitrary parameter for illustrating
    | | function syntax and documentation
    Return:
    - (int) arbReturn - an arbitrary return value
    */

    int arbReturn = 0;
    return arbReturn;
}
```

D. Control and Flow

There are different types of **control structures** we can use to tell the code under what conditions it should do certain things and how many times it should do those things. Before we dive into the major control structures, I want to cover a few keywords.

1. break

The **break** keyword is used to exit from a loop or a switch case statement. If you want to get out of some structure, especially a loop, but you have yet to meet the exit conditions, this is when you use break. You'll get lots of practice with break in section F.

2. continue

The **continue** keyword skips the rest of the code in whatever loop is being executed, and then the condition for the loop is checked to see whether the loop must be executed again.

3. goto

You are unlikely to ever see the **goto** keyword in the wild, because many C language programmers frown upon its use. Despite that, I think its interesting, so I'm going to tell you about it⁴. When you use goto, it redirects the flow of the code to a specified label.

E. if, else if, else

This control structure is used for determining a course of action based on conditions of varying complexity.

```
if (condition) {
    // do something
}

else if (condition) {
    // do something
}

else {
    // do something
}
```

⁴ I bet the C-programmers are C-thing. Haha (get it... like seething??)

F. switch... case

Switch case is similar to if else, but the conditions it evaluates are far simpler. Switch case looks at ONE variable and performs different actions based on the value of that variable.

```
switch (<variable being examined>) {
    case <value 1>:
        // do something
        break;
    case <value 2>:
        // do something
        break;
    default:
        // what to do if none of the cases are true
        break;
}
```

G. while and do while

A **while loop** will perform the code inside of it over and over until a certain condition becomes false. The condition is evaluated before the code in the loop is executed.

```
while (condition) {
    // do something
}
```

It is very important that you **UPDATE THE CONDITION** in the loop, or the while loop will continue forever, and you will be stuck in an **infinite loop**.

A **do while loop** differs from a while loop in that its condition is evaluated after the code in the loop is executed. Consequently, the code in a do while loop will always be executed at least once.

```
do {
    // do something
} while (condition);
```

H. For

Similar to the while loop, a **for loop** will repeat while a certain condition remains true. However, when you create a for loop, you must initialize a variable to be used in your condition and specify how that variable will be updated each loop.

```
for (initialization; condition; increment) {
    // do something
}
```